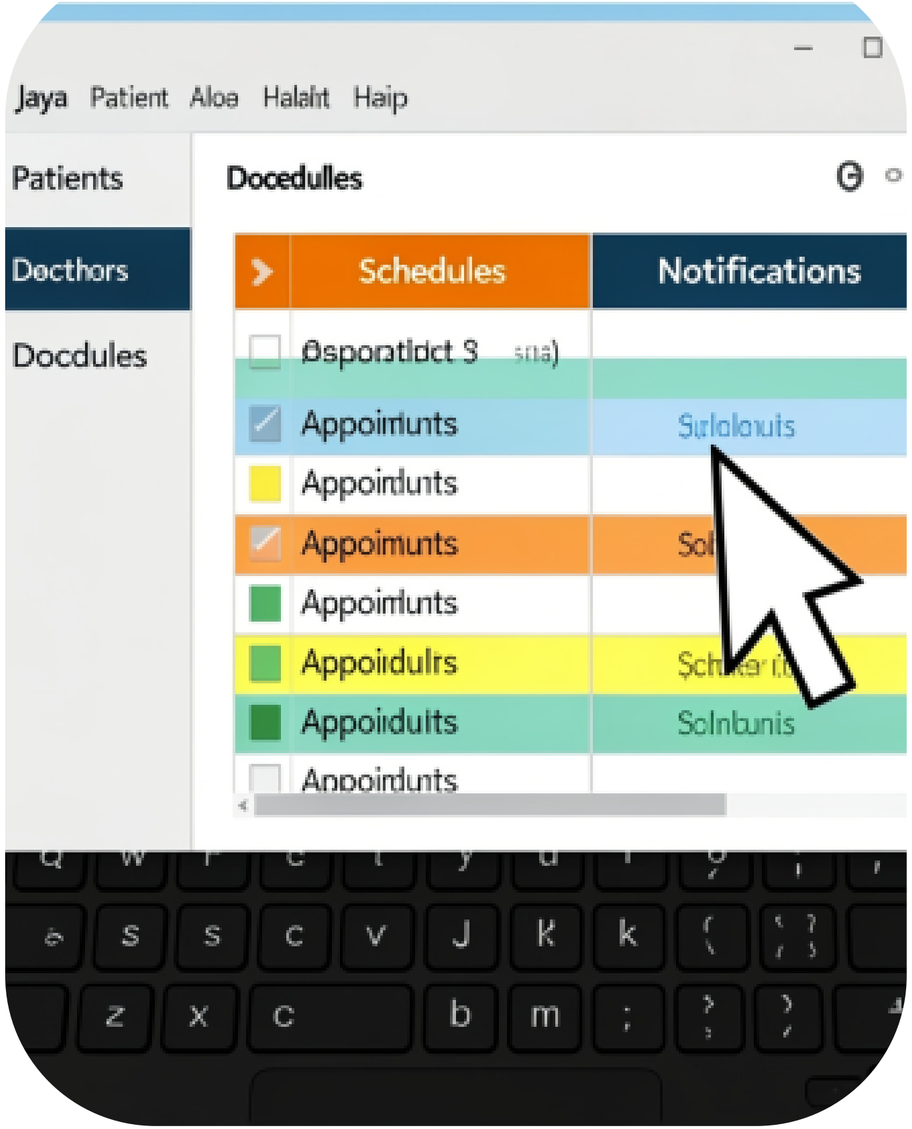
MediCare+

Hospital appointment booking system
with OOP and design patterns

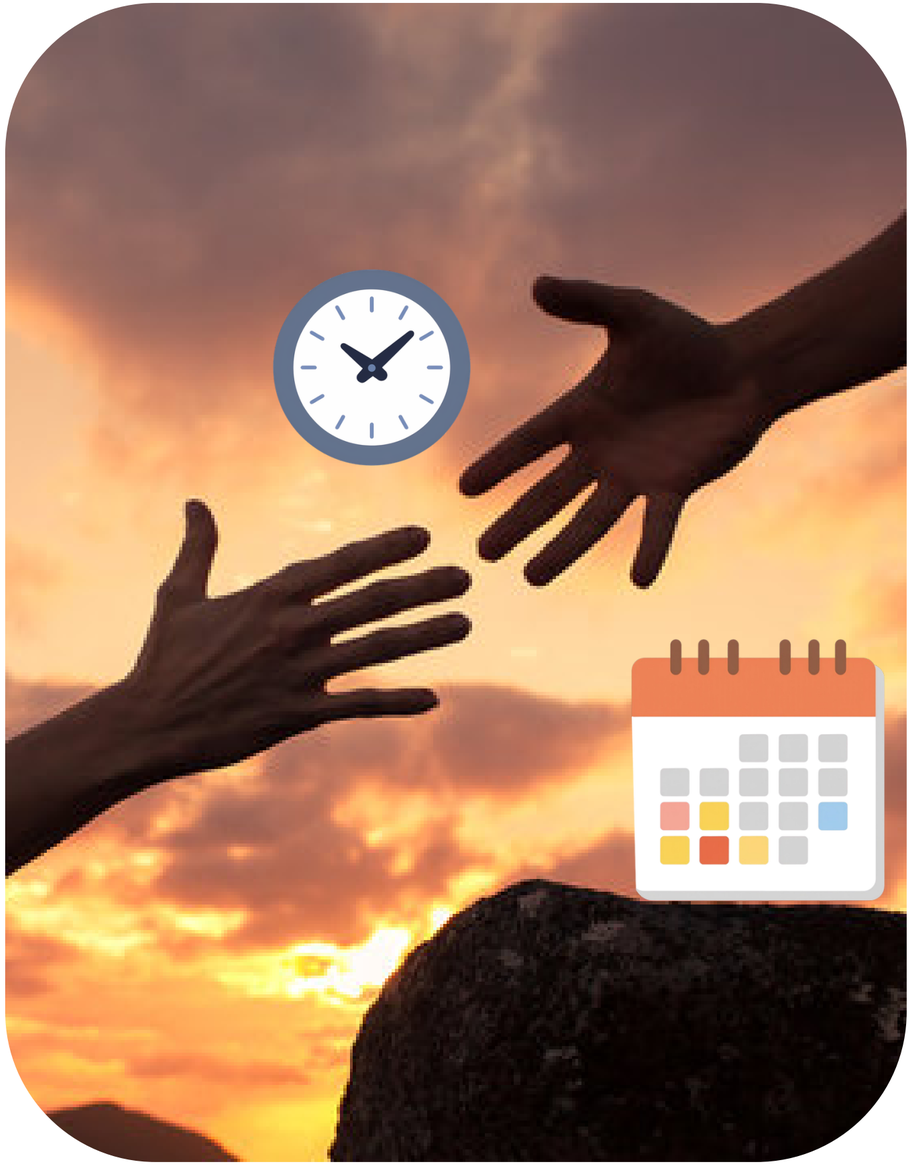
Introduction

MediCare+ is a desktop Java application that digitizes hospital appointment management, reducing missed and double-booked appointments.

The system combines *object-oriented programming* with three design patterns to automate scheduling, notifications, and record-keeping for patients and clinicians.



01 Program Overview



OUR PURPOSE

- Platform managing an appointment schedule effectively

NOTIFICATION | REAL TIME | RECORD

The Problem in Today's Hospital Systems

Current Problems

- Long waiting times
- Double bookings
- Manual scheduling errors
- Missed appointments
- Uninformed doctors

With MediCare+:

- Real-time scheduling
- No double bookings
- Automated system
- Instant notifications
- Notified and informed Doctors

Case Scenario

Booking an Appointment

- Fast and simple booking, anytime you need care.

Case Scenario

Cancelling or Rescheduling

- Flexible scheduling that adapts to your time.

Case Scenario

Searching for a Doctor

- Find the right specialist in seconds.

Case Scenario

Viewing Appointment Dashboard

- All your appointments, clear and organized in one place.

Case Scenario

Notification of Appointment Changes

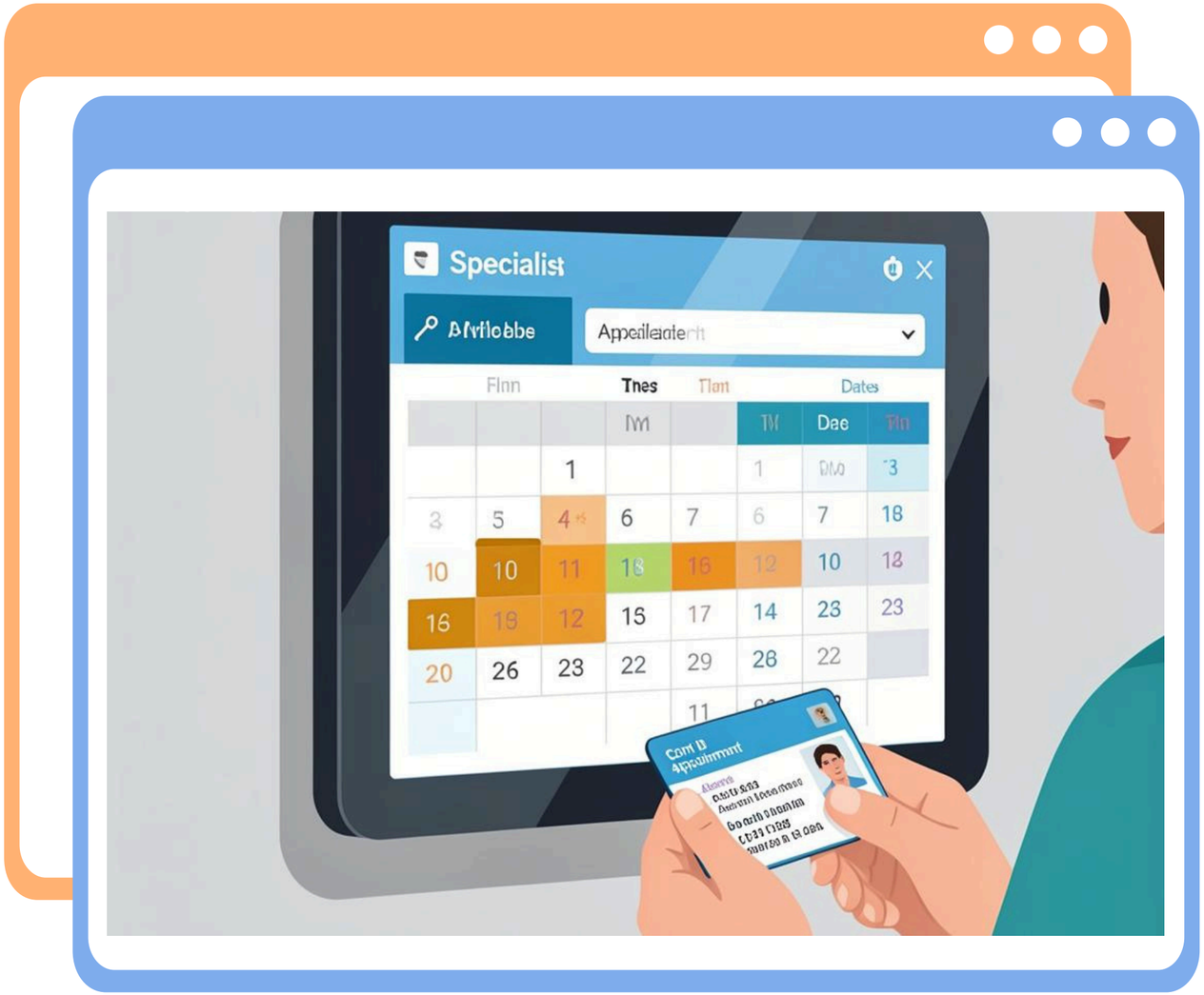
- Stay informed instantly; never miss an update.

Key Features

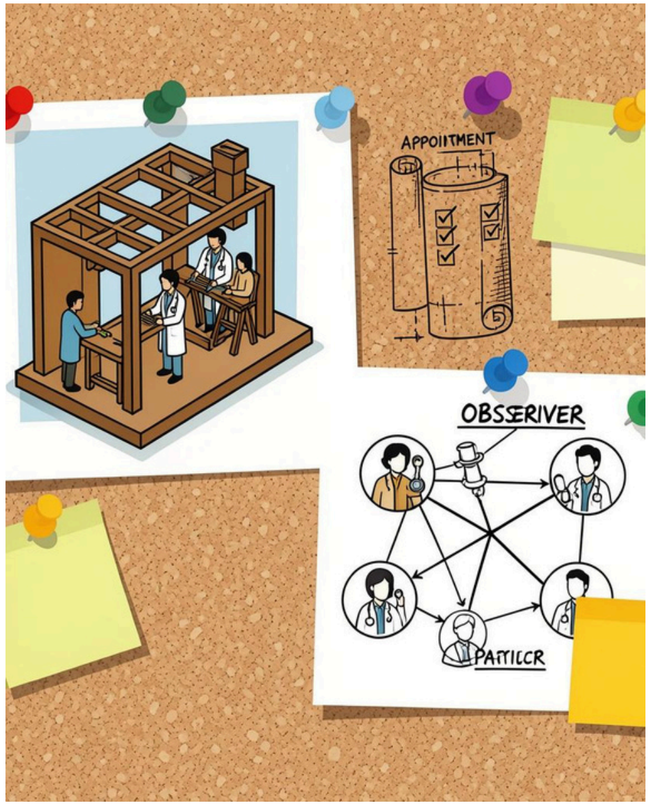
- Intuitive patient dashboard, searchable doctor directory, and flexible booking flow.
- Uses **Observer** for notifications, **Factory** for specialty creation, and **Builder** for validated appointment assembly.

User Workflow

- **Log in** to view a personalized dashboard with appointments and recommendations
- **Search and book** specialists quickly with pre-filled patient details
- **Manage appointments** easily with instant updates and rescheduling



02
Technical Design



Design Patterns

- ***Factory*** creates doctor/specialty objects, enabling new specialties with minimal code changes.
- ***Builder*** assembles validated appointments step-by-step to ensure data integrity before confirmation.
- ***Observer*** dispatches notifications to subscribed parties when appointment status changes, ensuring synchronous updates.

PATTERN 1

Factory Pattern

Creational

Defines an interface for creating objects but lets subclasses decide which class to instantiate. In this system, `DoctorFactory` decouples doctor creation from the booking logic — adding a new specialty only requires a new factory and doctor class, with zero changes to existing code.

Abstract Creator: DoctorFactory | **Concrete Creators:** CardiologistFactory, DermatologistFactory, PediatricianFactory | **Abstract Product:** Doctor | **Concrete Products:** Cardiologist, Dermatologist, Pediatrician

```
// Abstract Creator — declares the factory method public abstract class DoctorFactory { public abstract Doctor createDoctor String name); } // Concrete Creator — decides which
product to instantiate public class CardiologistFactory extends DoctorFactory { public Doctor createDoctor String name) { return new Cardiologist name); } } // Usage in MainFrame —
no "new Cardiologist()" scattered in UI code DoctorFactory f = new CardiologistFactory(); Doctor d1 = f.createDoctor "Ramon Ramos" ;
```

PATTERN 2

Observer Pattern

Behavioural

Defines a one-to-many dependency so that when `AppointmentManager` changes state (booking or cancellation), all registered observers are notified automatically. New notification channels (e.g. SMS) can be added by implementing the observer interface — no changes to the manager.

Subject Interface: AppointmentSubject | **Observer Interface:** AppointmentObserver | **Concrete Subject:** AppointmentManager | **Concrete Observers:** PatientNotifier, DoctorNotifier

```
// Observer interface — the contract every notifier must fulfill public interface AppointmentObserver { void update Appointment appt, String status); } // Concrete subject fires
all observers on booking public void book Appointment appt) { appointments.add appt); notifyObservers appt, "CONFIRMED" ; // calls update() on each } // PatientNotifier reacts to
status change public void update Appointment a, String status) { System.out.println "[PATIENT] Appointment is now: " + status); } // DoctorNotifier reacts independently public void
update Appointment a, String status) { System.out.println "[DOCTOR] Schedule updated: " + status); }
```

PATTERN 3

Builder Pattern

Creational

Constructs a complex `Appointment` object step by step using method chaining. The `build()` method validates all required fields before returning the object, preventing incomplete or invalid appointments from being created.

Product: Appointment | **Builder:** Appointment.AppointmentBuilder (inner static class) | **Fields set:** patient, doctor, date, time, notes

```
// Builder — each setter returns "this" for chaining public static class AppointmentBuilder { public AppointmentBuilder setPatient Patient p) { appt.patient = p; return this; }
public Appointment build() { if (appt.patient == null || appt.doctor == null) throw new IllegalStateException "Missing required fields"); appt.status = "CONFIRMED"; return appt; }
} // Fluent usage in BookScreen — readable, step-by-step construction Appointment appt = new Appointment.AppointmentBuilder() .setPatient(patient) .setDoctor(selectedDoctor)
.setDate(LocalDate.parse(dateInput)) .setTime(LocalTime.parse(timeInput)) .setNotes(notesInput) .build();
```

OOP Principles

Encapsulation

All data fields are declared private. External classes access data only through public getter methods or the builder. Internal state cannot be modified directly from outside the class.

Example: Appointment fields (patient, doctor, date, time, notes, status) are all private. The constructor is private tooonly AppointmentBuilder can instantiate Appointment. Outsiders use getDoctor(), getDate(), getStatus() etc.

PATIENT.JAVA BUILDER/PATIENT.JAVA

```
public class Patient {
 private String name;  PRIVATE  private int age;  PRIVATE  private String patientId;
  PRIVATE
public Patient String name, int age, String id) { this.name = name; this.age = age;
this.patientId = id; }
 // Public getters — read only public String getName() { return name; }  GETTER
 public int getAge() { return age; }  GETTER  public String getId() { return
 patientId; }  GETTER
}
```

APPOINTMENT.JAVA BUILDER/APPOINTMENT.JAVA

```
public class Appointment {
 private Patient patient;  PRIVATE  private Doctor doctor;  PRIVATE  private LocalDate
 date;  PRIVATE  private LocalTime time;  PRIVATE  private String status;  PRIVATE
 private Appointment() {}  PRIVATE CONSTRUCTOR
 // Only getters — no setters
 public Patient getPatient() { return patient; } public Doctor getDoctor() { return
 doctor; } public LocalDate getDate() { return date; } public String getStatus() {
 return status; }
}
```

OOP Principles

Abstraction

Complex implementation is hidden behind abstract classes and interfaces. The caller only needs to know what a method does, not how it does it internally.

Example: Doctor is abstract-callers use getInfo() without knowing if it's a Cardiologist or Pediatrician. AppointmentObserver hides how PatientNotifier and DoctorNotifier each respond to the same notification.

DOCTOR.JAVA — ABSTRACT CLASS FACTORY/DOCTOR.JAVA

```
public abstract class Doctor {                    ABSTRACT
protected String name; protected String specialty; protected String email; protected
double rating; protected int yearsExp; // Caller knows this method exists // but NOT
how each subclass implements it
public abstract String getInfo();                 ABSTRACT METHOD
public String getName() { return name; } public String getSpecialty() { return
specialty; } public double getRating() { return rating; } }
```

APPOINTMENTOBSERVER.JAVA — INTERFACE OBSERVER/APPOINTMENTOBSERVER.JAVA

```
public interface AppointmentObserver {            INTERFACE
// Just the contract — no implementation
void update(Appointment appt, String status);     NO BODY
} // AppointmentManager only knows about // the interface — not the concrete classes
for (AppointmentObserver o : observers) {
o.update(appt, status);                           ABSTRACTED CALL
// Could be PatientNotifier // Could be DoctorNotifier // Manager doesn't care which
}
```

OOP Principles

Inheritance

Subclasses extend parent classes to reuse and specialise behaviour. Common fields and methods are defined once in the parent and shared by all child classes.

Example: Cardiologist, Dermatologist, and Pediatrician all extend Doctor, inheriting name, specialty, room, rating, and yearsExp. Each overrides getInfo() to provide its own implementation. Factories similarly extend DoctorFactory.

CARDIOLOGIST.JAVA EXTENDS DOCTOR FACTORY/CARDIOLOGIST.JAVA

```
public class Cardiologist
extends Doctor { INHERITS Doctor
public Cardiologist String name) {
// Calls parent constructor super(name, "Cardiology", "Room 204", 4.9, 12, true);
REUSES PARENT
}
@Override public String getInfo() { OVERRIDES return "Dr. " + name + " | " +
specialty + " | Rating: " + rating; }
}
```

PATIENTNOTIFIER.JAVA IMPLEMENTS APPOINTMENTOBSERVER OBSERVER/PATIENTNOTIFIER.JAVA

```
public class PatientNotifier
implements AppointmentObserver { IMPLEMENTS
private final String patientName; public PatientNotifier String n) {
this.patientName = n; }
@Override public void update Appointment a, String status) { FULFILLS CONTRACT
System.out.println( "[PATIENT] Dear " + patientName + ", your appointment is: " +
status); }
}
```

OOP Principles

Polymorphism

The same method call behaves differently depending on the actual object type at runtime. One interface, many implementations - chosen dynamically.

Example: notifyObservers() calls update() on every registered observer. PatientNotifier and DoctorNotifier each respond differently to the same call. createDoctor() returns any Doctor subtype depending on which factory is used.

APPOINTMENTMANAGER.JAVA — ONE CALL, TWO RESPONSES OBSERVER/APPOINTMENTMANAGER.JAVA

```
public void book Appointment appt) { appointments.add appt);
notifyObservers appt, "CONFIRMED");   ONE CALL
} public void notifyObservers Appointment a, String status) { for AppointmentObserver o : observers) {
o.update a, status);   POLYMORPHIC DISPATCH
// Java automatically calls the RIGHT update() // based on what "o" actually is at runtime } }
```

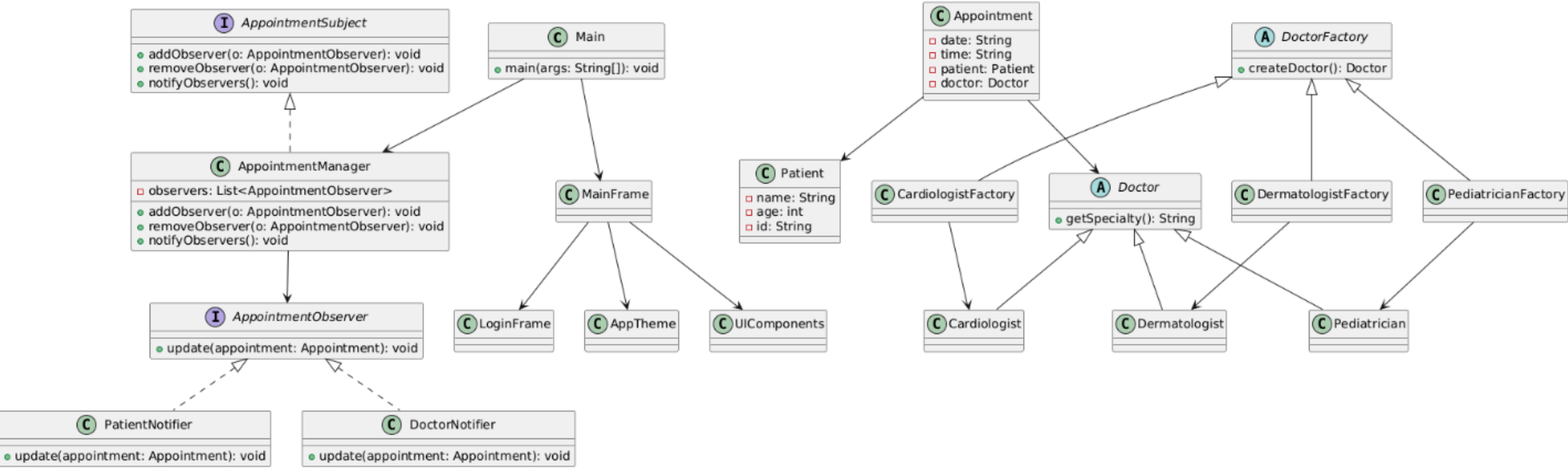
```
// When observers list has PatientNotifier + DoctorNotifier: o.update() → PatientNotifier.update() fires: [PATIENT] Dear Juan Dela Cruz, your appointment is: CONFIRMED o.update() →
DoctorNotifier.update() fires: [DOCTOR] Dr. Ramon Ramos, appointment with Juan is: CONFIRMED // Same method call — different behaviour — decided at runtime
```

MAINFRAME.JAVA — ONE DOCTOR REFERENCE, THREE POSSIBLE TYPES GUI/MAINFRAME.JAVA

```
// All stored as Doctor — their real type is hidden List Doctor doctors = Arrays asList
new CardiologistFactory() createDoctor "Ramon Ramos"),   Cardiologist new DermatologistFactory() createDoctor "Sara Santos"),   Dermatologist new
PediatricianFactory() createDoctor "Carlo Cruz")   Pediatrician
} // Loop calls getInfo() on each — same call, 3 different outputs for Doctor d : doctors) {
System.out.println d.getInfo());   POLYMORPHIC
}
```

```
// Each getInfo() is overridden differently in each subclass: Cardiologist.getInfo() → Dr. Ramon Ramos | Cardiology | Rating: 4.9 | 12 yrs exp Dermatologist.getInfo() → Dr. Sara Santos |
Dermatology | Rating: 4.7 | 8 yrs exp Pediatrician.getInfo() → Dr. Carlo Cruz | Pediatrics | Rating: 4.8 | 10 yrs exp // Same d.getInfo() call — 3 different results based on actual type
```

Class Diagram



Summary

- MediCare+ digitizes hospital booking to reduce missed and double-booked appointments while improving staff efficiency.

- The modular Java design and applied patterns enable scalability, reliable notifications, and simplified specialty management.